K J Somaiya Institute of Technology
An Autonomous Institute Permanently Affiliated to the University of Mumbai

DEPARTMENT OF INFORMATION TECHNOLOGY

Name of Student : Harshal Nitin Mehta
Roll No : B - 63
Batch : SB4

EXPERIMENT 2

Aim:

Implementation of queue using Array.

Objective:

- To implement a linear queue using an array.
- To perform insert, delete & display operations.
- To understand FIFO (First In First Out) behaviour.
- To handle overflow & underflow conditions.

Theory:

A queue is a linear data structure that follows the FIFO principle. This means the element inserted first is deleted first. A queue can be implemented using an array with two pointers front & rear.

In this experiment, a linear queue is implemented using an array. The FRONT and REAR variable are used to keep track of the first and last element of the queue.

The queue performs three basic operations:

- Enqueue
- Dequeue
- Display.

Enqueue is used to insert an element into the queue, Dequeue is used to remove an element from the queue and Display is used to show the elements present in the queue.

The program also handles overflow when an element is inserted into a full queue & underflow when an attempt is made to delete an element from an empty queue.

Program :

```
#include <stdio.h>

#define MAX 5


int queue[MAX];

int front = -1, rear = -1;


void enqueue(int value)

{

   if (rear == MAX - 1)

   {

      printf("Queue Overflow!\n");

   }

   else

   {

      if (front == -1)
```

```
            front = 0;

        rear++;
        queue[rear] = value;
        printf("%d inserted into queue.\n", value);
    }
}

void dequeue()
{
    if (front == -1 || front > rear)
    {
        printf("Queue Underflow!\n");
    }
    else
    {
        printf("%d deleted from queue.\n", queue[front]);
        front++;
    }
}

void display()
{
```

```
    int i;

    if (front == -1 || front > rear)
    {
        printf("Queue is empty.\n");
    }
    else
    {
        printf("Queue elements: ");
        for (i = front; i <= rear; i++)
        {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
```

```
    display();

    dequeue();

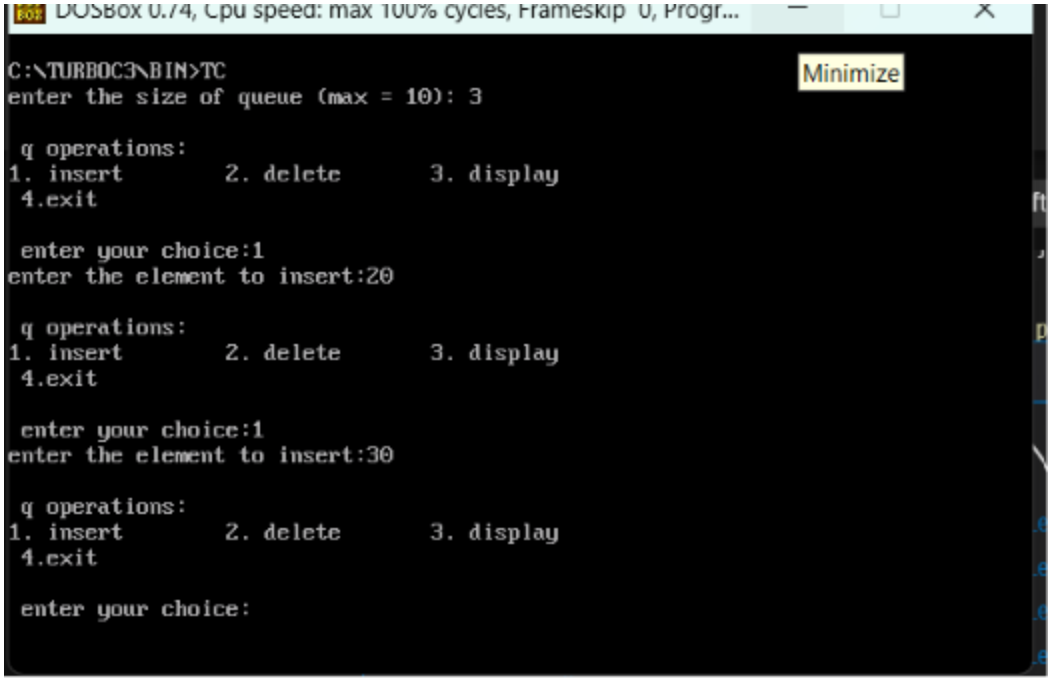
    display();

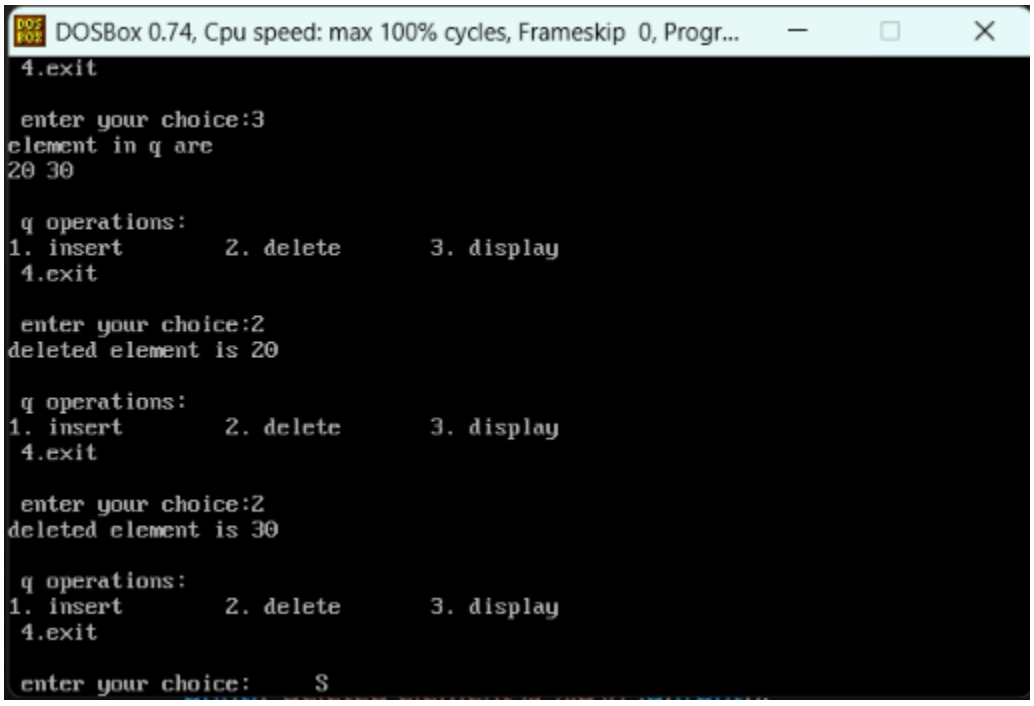
    enqueue(40);
    enqueue(50);

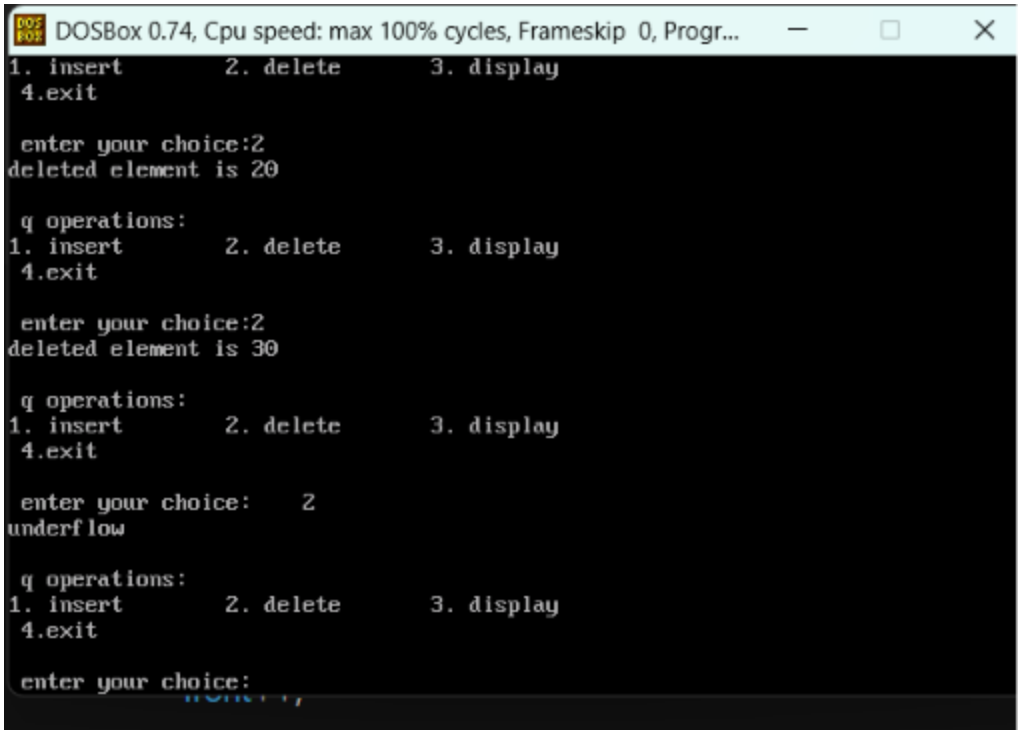
    display();

    return 0;
}
```

Output :


DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
Minimize
C:\TURBOC3\BIN>TC
enter the size of queue (max = 10): 3
q operations:
1. insert 2. delete 3. display
4.exit
enter your choice:1
enter the element to insert:20
q operations:
1. insert 2. delete 3. display
4.exit
enter your choice:1
enter the element to insert:30
q operations:
1. insert 2. delete 3. display
4.exit
enter your choice:


DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Progr...
4.exit
enter your choice:3
element in q are
20 30
q operations:
1. insert 2. delete 3. display
4.exit
enter your choice:2
deleted element is 20
q operations:
1. insert 2. delete 3. display
4.exit
enter your choice:2
deleted element is 30
q operations:
1. insert 2. delete 3. display
4.exit
enter your choice: S



Outcome:

The linear queue was successfully implemented using an array. The Insert, Delete and Display operations were performed according to the FIFO principle. Overflow & Underflow conditions were also handled successfully.

Conclusion:

The program successfully demonstrates the implementation of linear queue using an array & the working of its basic operations along with overflow & underflow conditions.

Algorithm:

1. Start
2. Initialize front = rear = -1
3. Enqueue: Check if rear == MAX-1.
   If not, increment rear & insert element.

4. Dequeue: Check if front == -1 or front > rear.
   If not, delete queue[front] and increment front.
5. Display: Print elements from front to rear.
6. Repeat operations until Exit.
7. Stop.